Qué cambió en `problema_1_benja.py` y cómo influye sobre el original

`problema_1.py` quedó intacto. `problema_1_benja.py` es una copia con 4 cambios, y guarda sus figuras en `images/output_benja/`. Este documento responde dos preguntas: **¿cambia algo?** y **¿lo complejiza?**. También explica **HoughCircles** y por qué no se usó.

Respuesta corta

- **Resultado final**: idéntico. 19 objetos, 9 monedas de 10c, 5 de 1 peso, 3 de 50c, dados 5 y 3.
- **Segmentación**: los 19 objetos se detectan igual, pero el área de algunos cambia entre 1% y 2% (por ejemplo, un dado pasa de 50386 a 49272 px y una moneda plateada de 55531 a 55108 px). Es por el umbral de brillo 194 contra 190.
- **Complejidad**: sube. El archivo pasa de 388 a 412 líneas (56 líneas nuevas y 32 modificadas o quitadas). Entran dos conceptos nuevos para explicar: k-means y Otsu sobre un subconjunto de píxeles.
- **Mejora real**: una sola es visible, la clausura en los puntos de los dados. El resto no cambia nada en esta imagen y solo podría ayudar con otras fotos, que no se probaron.

Cambio 1: umbral de brillo con Otsu (etapa A, segmentación)

- **Qué hace**: en lugar de fijar `UMBRAL_BRILLO = 190`, Otsu elige solo el corte entre "objeto claro" y "fondo". Da 194.
- **Cómo influye en la segmentación**: cambia ligeramente el borde de los objetos claros (monedas plateadas y dados), por eso sus áreas varían 1% a 2%. No pierde ni agrega objetos.
- **Condición necesaria**: Otsu se calcula solo con los píxeles claros (`L > 150`, nueva constante `LIMITE_CLARO`). Con toda la imagen el fondo se mezcla con los objetos y la segmentación falla.
- **Complejidad**: sube un poco. Se cambió un número fijo (190) por otro (150) más una llamada a Otsu. El número fijo no desapareció.
- **Veredicto**: no mejora. Conviene dejarlo fuera.

Cambio 2: k-means para clasificar monedas (etapa B)

- **Qué hace**: en lugar de dos umbrales de área escritos a mano (70000 y 95000), k-means agrupa las áreas en 3 grupos. Los centros que encuentra son 58090, 85496 y 105682 px.
- **Cómo influye**: la clasificación es idéntica. Se gana que, si cambia el tamaño de las monedas en la foto, no hay que reajustar números.
- **Complejidad**: sube. Una función de unas 20 líneas (con semilla fija para que sea reproducible) reemplaza una de 6 líneas con dos constantes.
- **Veredicto**: neutro. Sirve para mostrar que se usó la U5. Si solo importa que funcione, el original es más simple.

Cambio 3: área mínima del pip relativa (etapa C, dados)

- **Qué hace**: antes un pip de la cara superior debía tener más de 1000 px. Ahora debe medir al menos la mitad del pip redondo más grande de ese mismo dado.
- **Cómo influye**: en esta imagen cuenta los mismos pips (5 y 3). Sería útil si la foto se toma a otra distancia.
- **Complejidad**: sube poco (3 líneas nuevas).
- **Veredicto**: neutro.

Cambio 4: clausura en los pips (etapa C, dados)

- **Qué hace**: aplica una clausura de 15x15 sobre los píxeles oscuros del dado. Tapa los reflejos blancos y las mordidas del borde de cada pip.
- **Cómo influye**: en la figura C1 los cinco pips quedan como círculos completos, antes tenían agujeritos. El conteo no cambia.
- **Complejidad**: casi nada. Una línea y una constante.
- **Veredicto**: es la mejor del grupo.

HoughCircles: qué hace y por qué no se usó

- **Qué hace, en simple**: busca círculos en la imagen. Primero detecta los bordes. Después cada punto del borde "vota" por los centros de círculo posibles. Donde se juntan más votos hay un círculo. Devuelve la posición del centro y el radio de cada uno.
- **Se probó sobre `monedas.jpg`** (imagen reducida a un cuarto, con filtro de mediana):
  - Con un parámetro de votos (`param2`) de 30 a 40, detecta los 17 círculos que son las monedas, ninguno en los dados.
  - Por radio se separan en tres grupos limpios: 9 de 10c, 5 de 1 peso y 3 de 50c. Es el mismo resultado que el programa actual.
  - Es rápido: se ejecuta en centésimas de segundo.
- **Por qué no se usó igual**:
  - **Es sensible a sus parámetros.** Con `param2 = 20` encuentra 20 círculos (3 de más). Con 50, 15 (se pierden 2). Con 60, 14. Cambiar `dp` o `param1` también mueve el total entre 14 y 21. Hay que ajustarlo a mano para cada foto.
  - **Solo da círculos.** El Punto A de la consigna pide segmentar monedas **y dados**. Hough no dibuja la forma de cada objeto y no puede con los dados, que son cuadrados. Seguiría haciendo falta la segmentación por color para los dados.
  - **El radio es un poco menor que el real** (por ejemplo 124 contra unos 136 medido en la máscara). No afecta a la clasificación, pero se perdería la medición de área exacta por contorno.
- **Qué sí podría aportar**: una verificación cruzada. Si Hough y la máscara coinciden en 17 monedas y en sus tamaños, el resultado es más confiable. Como alternativa total al método actual no conviene.

Archivos

- `problema_1.py`: original, sin cambios.
- `problema_1_benja.py`: versión con los cuatro cambios.
- `images/output_benja/`: figuras de la versión nueva.